

Uso responsable de IA

Durante el desarrollo de QLess se utilizó asistencia con herramientas de IA como apoyo técnico para acelerar la investigación, comparar alternativas y resolver problemas puntuales de implementación. La IA no reemplazó el trabajo del equipo: todo el código incorporado fue revisado, adaptado y probado por los integrantes antes de integrarse al repositorio.

Declaración general de revisión humana

Todas las respuestas obtenidas mediante IA fueron utilizadas como apoyo y no como entrega directa. El equipo revisó la coherencia técnica, corrigió errores de dependencias, adaptó el código al proyecto, probó los flujos principales en Android Studio, ejecutó merges y verificó que las funcionalidades respondieran a los requisitos del trabajo práctico.

Manejo de secretos y configuración sensible

Al inicio del proyecto se habían cargado claves (contraseñas y tokens de distintas plataformas) directamente en el código. El equipo lo identificó como una mala práctica de seguridad: las removió del repositorio, rotó las credenciales expuestas y movió la configuración sensible a `local.properties` y `BuildConfig`, fuera del control de versiones. La IA se usó para confirmar buenas prácticas de manejo de secretos, pero la detección, la corrección y la rotación las hizo el equipo.

Prompts usados

La herramienta de IA utilizada fue Claude Code (asistente de IA en la terminal). Los prompts que siguen son reconstrucciones representativas de las consultas realizadas, no transcripciones literales.

1. Elección de backend y persistencia

Prompt utilizado/reconstruido:

Estamos desarrollando una app Android nativa en Kotlin con Jetpack Compose para pedidos en locales gastronómicos. Necesitamos autenticación de usuarios, persistencia de locales, productos y pedidos, y posibilidad de sincronizar datos en tiempo real para un panel administrativo. ¿Qué opciones de backend convienen para un MVP académico y cuál recomendarías considerando facilidad de integración, autenticación, base de datos y escalabilidad?

Resultado generado por IA:

La IA sugirió comparar Firebase, Supabase y un backend propio. Se comparó las distintas opciones y sus pros y contras. Se recomendó Supabase porque permite autenticación, base de datos PostgreSQL, APIs automáticas, realtime y buena integración para un MVP.

Fragmentos o ideas generadas:

- Comparación entre Firebase, Supabase y un backend propio.
- Supabase recomendado por autenticación, PostgreSQL, APIs automáticas y realtime.
- Separar la configuración sensible del código.

Revisión humana realizada:

El equipo evaluó la recomendación contra los requisitos del TP y decidió usar Supabase por su equilibrio entre velocidad de implementación, backend gestionado y soporte para autenticación y persistencia remota. Se descartó subir claves directamente al repositorio y se separó la configuración sensible.

2. Integración de Google Auth con Supabase

Prompt utilizado/reconstruido:

Necesito integrar inicio de sesión con Google en una app Android Kotlin con Jetpack Compose usando Supabase como backend. Explicame qué configuración necesito en Google Cloud, qué datos van en Supabase y cómo debería estructurar el flujo de login sin exponer secretos en el código.

Resultado generado por IA:

La IA ayudó a identificar la necesidad de configurar OAuth en Google Cloud, obtener el Web Client ID, configurar el proveedor Google en Supabase y adaptar el flujo de login dentro de la app.

Fragmentos o ideas generadas:

- Separar credenciales y configuración sensible.
- Usar placeholders para URL, anon key y client ID.
- Crear un caso de uso específico para login con Google.
- Mantener la lógica de autenticación fuera de la pantalla Compose.

Revisión humana realizada:

El equipo configuró manualmente Google Cloud y Supabase, revisó los errores de Gradle/dependencias, ajustó los imports y validó el flujo de inicio de sesión en Android Studio.

3. Implementación de CameraX y escaneo QR

Prompt utilizado/reconstruido:

En una app Android con Jetpack Compose necesito usar la cámara para leer códigos QR de locales. Quiero usar CameraX y ML Kit Barcode Scanning. ¿Cómo puedo organizar la implementación para abrir la cámara, detectar el QR y navegar a la pantalla del local si el código pertenece a la app?

Resultado generado por IA:

La IA propuso usar CameraX para la vista previa de cámara y ML Kit Barcode Scanning para detectar códigos QR. También sugirió validar el contenido del QR contra los locales registrados y manejar un flujo de error cuando el QR no sea reconocido.

Fragmentos o ideas generadas:

- Pantalla de escaneo QR con cámara nativa.
- Validación del contenido escaneado.
- Navegación hacia el detalle del local.
- Pantalla de error para códigos no pertenecientes a QLess.

Revisión humana realizada:

El equipo adaptó la solución a la navegación existente de la app, revisó permisos de cámara, probó el escaneo con QRs generados para locales de prueba y ajustó el comportamiento para evitar múltiples lecturas consecutivas del mismo código.

4. Integración de Google Maps y ubicación del usuario

Prompt utilizado/reconstruido:

Necesito mostrar locales cercanos al usuario en una app Android. La app usa Jetpack Compose y debería obtener la ubicación actual, calcular distancia contra locales guardados en la base y permitir ordenar por cercanía. Si el usuario está dentro de 50m o menos de un local, debería activar la pantalla de “Estas acá”. ¿Qué arquitectura y componentes conviene usar?

Resultado generado por IA:

La IA recomendó usar Google Play Services Location para obtener la ubicación del usuario, calcular distancia entre coordenadas y ordenar los locales por cercanía. También sugirió mostrar un destacado en la home con el local más cercano.

Fragmentos o ideas generadas:

- Servicio de ubicación separado de la UI.
- Cálculo dinámico de distancia en metros o kilómetros.
- Ordenamiento “el más cercano”.
- Banner destacado para el local más cercano.

- Integración opcional con Google Maps para visualizar ubicación.

Revisión humana realizada:

El equipo revisó permisos de ubicación, adaptó el cálculo de distancia al modelo de datos de locales y validó que la UI no dependa directamente del proveedor de ubicación, manteniendo la separación por capas.

5. Paleta de colores, Material 3 y modo oscuro

Prompt utilizado/reconstruido:

Tenemos una app gastronómica llamada QLess y queremos una identidad visual cálida. Ya tenemos colores para el modo claro, pero necesitamos una paleta diferenciada para modo oscuro que mantenga coherencia con Material Design 3. Proponé colores principales, secundarios y de superficie, considerando accesibilidad y contraste.

Resultado generado por IA:

La IA propuso una paleta basada en colores cálidos y gastronómicos, como Pimentón, Espresso y Albahaca, adaptados para modo claro y modo oscuro.

Fragmentos o ideas generadas:

- Color principal asociado a identidad gastronómica.
- Superficies oscuras con contraste suficiente.
- Colores secundarios para acciones y estados.
- Adaptación a Material 3.

Revisión humana realizada:

El equipo revisó visualmente la paleta en pantallas reales, ajustó contraste, estados de botones, cards y textos, y validó que el tema oscuro no fuera simplemente una inversión del tema claro.

6. Navegación y menú inferior personalizado

Prompt utilizado/reconstruido:

En Jetpack Compose necesito crear una barra de navegación inferior personalizada con varias secciones y un botón central de QR más destacado que sobresalga visualmente. ¿Cómo puedo estructurarlo usando Box, Row, zIndex y Navigation Compose?

Resultado generado por IA:

La IA sugirió usar un contenedor Box para superponer el botón central, una Row para los ítems de navegación y zIndex para asegurar que el botón QR quede por encima del resto de la barra.

Fragmentos o ideas generadas:

- Bottom navigation custom.
- Botón central para escaneo QR.
- Uso de Box para superposición.
- Separación entre componente visual y navegación real.

Revisión humana realizada:

El equipo adaptó el diseño al prototipo de Figma, ajustó tamaños, sombras, espaciados y comportamiento de navegación para que el botón QR mantenga prioridad visual sin romper la usabilidad.

7. Pruebas automatizadas e integración en CI

Prompt utilizado/reconstruido:

Tenemos una app Android en Kotlin con Jetpack Compose y arquitectura MVVM por capas (casos de uso + repositorios con contratos). Queremos agregar pruebas que validen la lógica de la aplicación, sobre todo los ViewModels y casos de uso del flujo de pedidos, sin depender de la red ni de un dispositivo, y además correr pruebas de UI sobre componentes Compose. ¿Cómo conviene estructurar las pruebas unitarias e instrumentadas, y cómo las integramos en un pipeline de CI en GitHub Actions para que corran en cada push?

Resultado generado por IA:

La IA propuso separar pruebas unitarias JVM (ViewModels y casos de uso, con repositorios fake y un dispatcher de test) de pruebas instrumentadas de Compose, y sugirió un workflow de GitHub Actions que compile, corra lint y ejecute los tests en cada push y pull request.

Fragmentos o ideas generadas:

- Repositorios fake en lugar de un framework de mocking, para tests deterministas.
- Regla de test para reemplazar el dispatcher principal de corrutinas.
- Tests de los ViewModels del dominio principal (pedidos): carga, filtros, checkout y cambios de estado.
- Test de un composable stateless con Compose Testing.
- Workflow de CI (build + lint + unit tests) disparado por push y pull request.

Revisión humana realizada:

La decisión de incorporar integración continua con lint y pruebas en cada push fue del equipo; la IA solo ayudó a redactar el workflow de GitHub Actions. El equipo eligió además usar fakes propios en vez de MockK para no sumar dependencias y mantener los tests deterministas, escribió y adaptó los tests al modelo real de datos, y ejecutó testDebugUnitTest y connectedDebugAndroidTest en Android Studio y emulador verificando que pasaran (33 unitarios + 2 instrumentados). La evidencia de ejecución se adjunta en H2/test_results/.

8. Métricas no funcionales (cold start y fluidez)

Prompt utilizado/reconstruido:

La consigna nos exige medir el cold start (objetivo < 2,5 s) y la fluidez del scroll (> 54 fps) en un dispositivo de referencia. Tenemos la app Android instalable por adb. ¿Cómo podemos medir estas dos métricas de forma reproducible, sin montar un módulo de benchmark complejo, y de dónde sacamos los números crudos para documentarlos como evidencia?

Resultado generado por IA:

La IA explicó que el cold start se mide con am start -W (campo TotalTime) forzando un arranque en frío con force-stop previo, y que la fluidez/jank se obtiene con dumpsys gfxinfo (porcentaje de janky frames) tras hacer scroll. Propuso encapsular ambas mediciones en un script de shell por adb, como alternativa liviana a un módulo Macrobenchmark.

Fragmentos o ideas generadas:

- Script por adb con dos mediciones: cold start y jank.
- Force-stop antes de cada arranque para garantizar medición en frío.
- Promedio sobre varias iteraciones de cold start.
- Umbrales de la consigna como referencia (< 2,5 s y jank < 10%).

Revisión humana realizada:

El equipo decidió quedarse con el script por adb por ser simple y reproducible para el alcance del TP. Adaptó el script al package de la app, corrió las mediciones y guardó las salidas crudas como evidencia.

Decisiones donde mantuvimos criterio propio

No todo lo sugerido por la IA se incorporó tal cual. Algunos ejemplos donde el equipo decidió en contra o ajustó la recomendación:

- Dynamic color: descartamos el color dinámico de Material 3 (que adapta la paleta al fondo de pantalla del usuario) para preservar la identidad de marca gastronómica con una paleta fija.
- Pruebas: usamos repositorios fake propios en lugar de un framework de mocking, priorizando tests deterministas y sin dependencias extra.
- Métricas: medimos con un script por adb en vez de montar un módulo Macrobenchmark, por simplicidad y reproducibilidad para el alcance del TP.

Conclusión

La IA fue una herramienta para acelerar la investigación, comparar alternativas y resolver problemas puntuales de implementación, pero siempre con criterio humano detrás: las decisiones de arquitectura, la escritura y ejecución de las pruebas, la medición de métricas y la validación final del producto fueron del equipo.